

Navigation in SwiftUI

Navigation Stack and Navigation Link

What is Navigation?

Navigation allows users to move between different screens (views) in an app. Think of it like moving between pages in a book or rooms in a house.

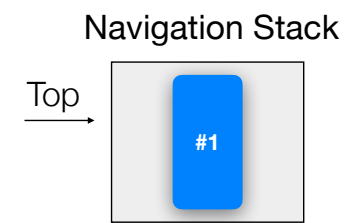
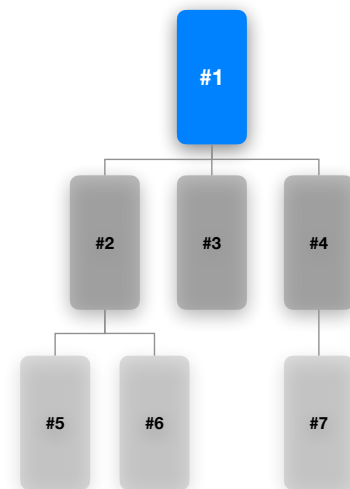
Real-world examples:

Settings app: Main screen → Wi-Fi settings → Network details

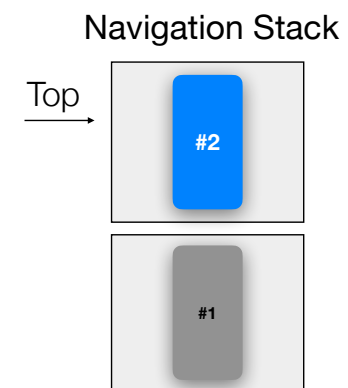
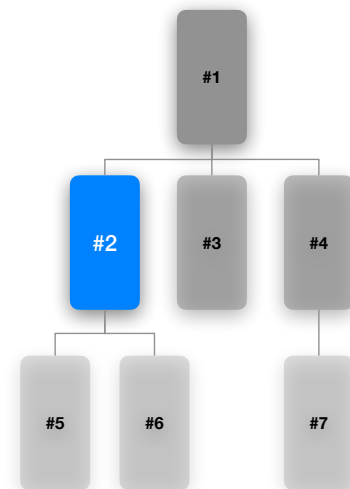
E-commerce app: Product list → Product details → Reviews

Social media: Feed → User profile → Photo detail

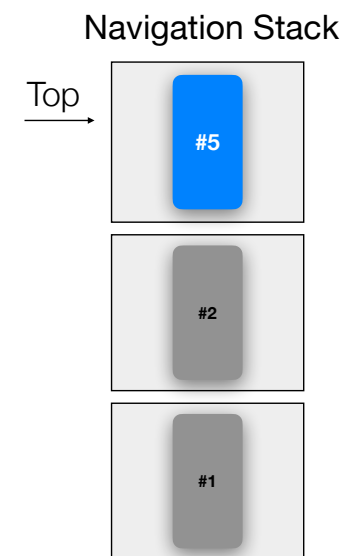
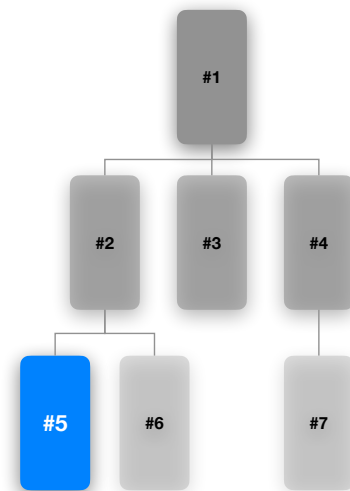
Stack in hierarchical navigation



Stack in hierarchical navigation



Stack in hierarchical navigation



SwiftUI Navigation Evolution

- iOS 13 - 15 **NavigationView** (deprecated)
- iOS 16+ **NavigationStack** (modern approach)

Its imperative to ensure we dont use the Navigation View to implement navigation especially given the fact that AI until recently is more likely to generate code using the deprecated APIs since that's what its trained on.

Navigation Stack

View

S `NavigationStack`

I `NavigationStack(root: () -> View )`

I `NavigationStack(path: Binding<NavigationPath> ,
root: () -> View)`

This session is focused on using only the 1st initializer - `NavigationStack(root: () -> View)`

Navigation Stack

API

```
| NavigationStack(root: () -> View )
```



```
| NavigationStack {  
    code  
}
```

This initializer takes 1 argument called **root** - which refers to the root of the stack - i.e. the 1st screen in the tree hierarchy of screens.

This closure takes no arguments and should return a view.

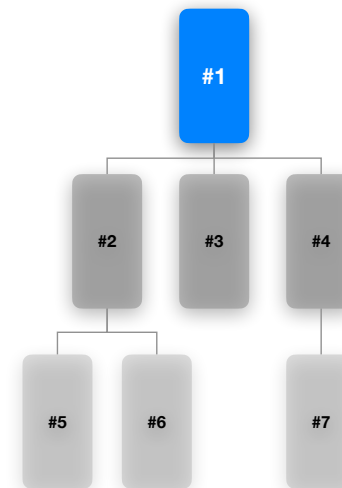
We can easily send this argument called **root** as a trailing closure into the initializer.

Navigation Stack

View

```
NavigationStack {  
  code  
}
```

- 1 stack for the full tree hierarchy.



NavigationStack: Container that enables navigation

Navigation Stack

Example

```
struct ContentView: View {  
    var body: some View {  
        NavigationStack {  
            VStack {  
                Image(systemName: "globe")  
                    .imageScale(.large)  
                    .foregroundStyle(.tint)  
                Text("Hello, world!")  
            }  
        }.padding()  
    }  
}
```

Preview does not change.

Navigation Title

View modifiers in Navigation Stack

`.navigationTitle( title: Text )`

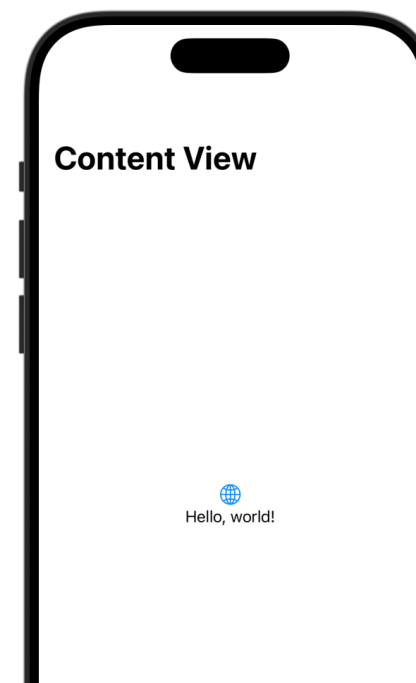
- View modifier available inside Navigation Stack.
- Not on/outside Navigation Stack

`.navigationTitle()`: Sets the title in the navigation bar

Navigation Title

Example

```
struct ContentView: View {  
    var body: some View {  
        NavigationStack {  
            VStack {  
                Image(systemName: "globe")  
                    .imageScale(.large)  
                    .foregroundColor(.tint)  
                Text("Hello, world!")  
            }  
        }.padding()  
    }.navigationTitle("Content View")  
}
```



Navigation Link

View

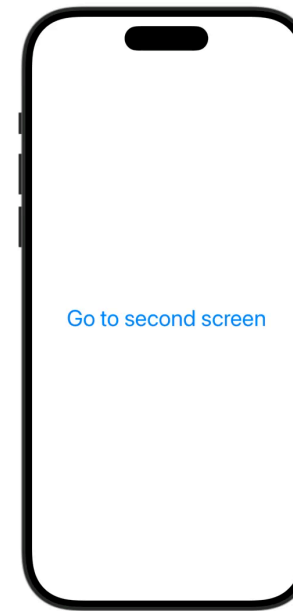
```
NavigationLink( titleKey: LocalizedStringKey ,  
                destination: () -> View )
```

1st API for Navigation link we are studying in this session is the one that takes arguments **title** & **destination**

Navigation Link

Example

```
struct ContentView: View {  
    var body: some View {  
        NavigationStack {  
            NavigationLink("Go to second screen",  
            destination: { SecondView() })  
                .font(.largeTitle)  
        }  
    }  
}  
  
struct SecondView: View {  
    var body: some View {  
        Text("This is the second view")  
            .font(.largeTitle)  
    }  
}
```



Navigation Link

API

```
NavigationLink( titleKey: LocalizedStringKey ,  
                destination: () -> View )
```



```
NavigationLink( titleKey: StringProtocol ) {  
    code  
}
```

1st API for Navigation link we are studying in this session is the one that takes arguments **title & destination**
Since this API has the last argument of a closure we can use trailing closure syntax to pass the destination view

Navigation Link

API

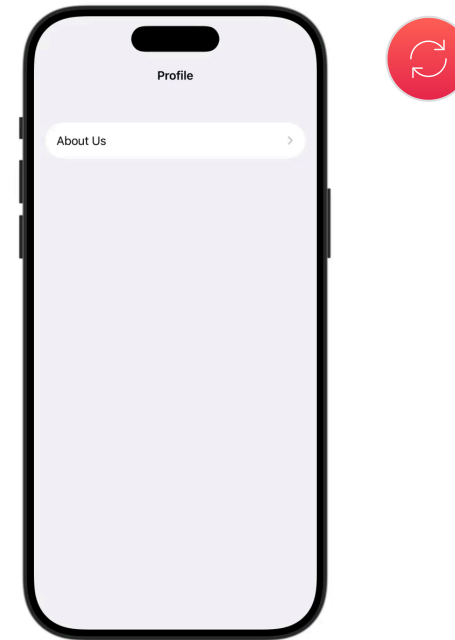
```
struct ContentView: View {  
    var body: some View {  
        NavigationStack {  
            NavigationLink("Go to second screen") {  
                SecondView()  
            }  
        }  
        .font(.largeTitle)  
    }  
}
```




Activity

ProfileApp

- **Hint**
- Use navigation link inside List.
- Observe the change in the list item.



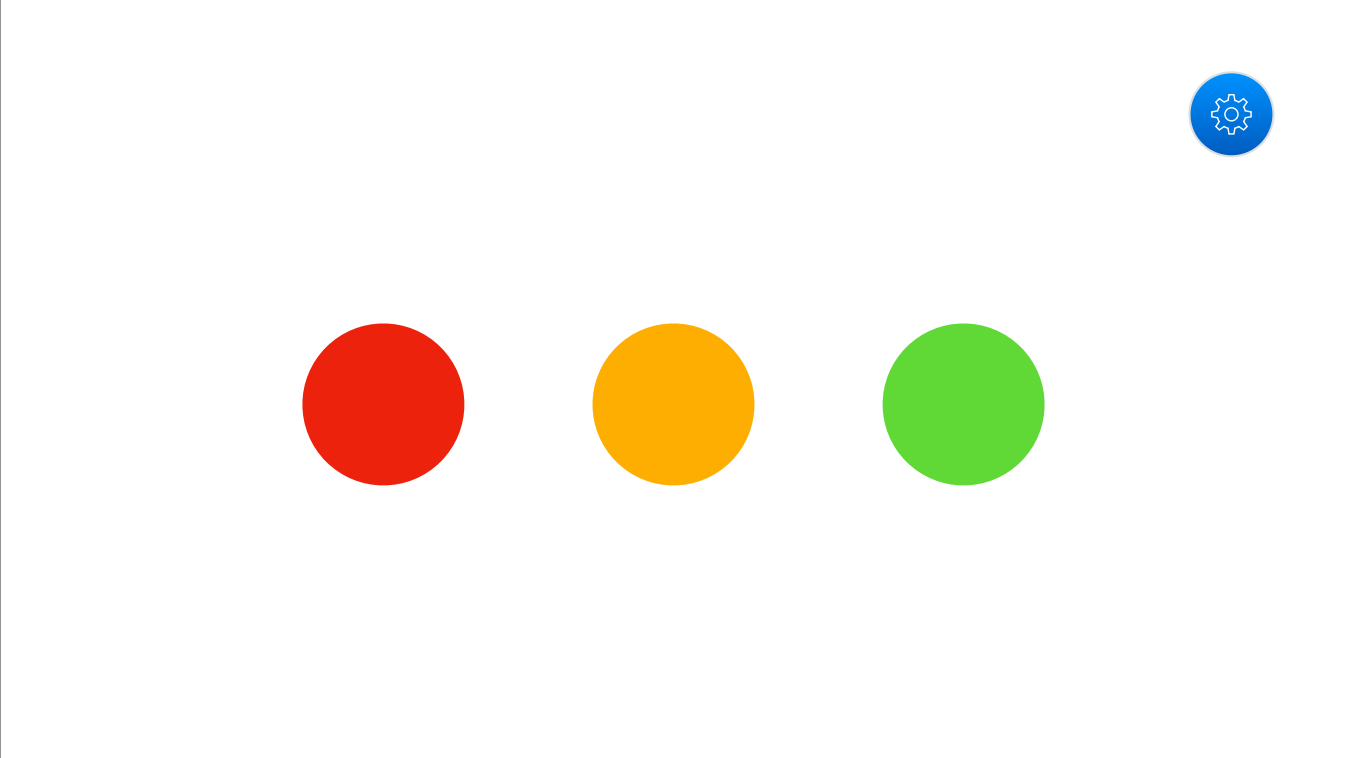
```
struct ProfileView: View {
  var body: some View {
    NavigationStack {
      List {
        NavigationLink("About Us", destination: AboutUsView())
      }
      .navigationTitle("Profile")
      .navigationBarTitleDisplayMode(.inline)
    }
  }
}
```

```
struct AboutUsView: View {
  var body: some View {
    List {
      ZStack {
        Color.mint
        Image(systemName: "graduationcap.fill")
          .font(.system(size: 60))
      }
    }
  }
}
```

```
}  
.frame(height: 200)  
.padding()  
Text("We are a team of 3 developers who met in a college canteen while avoiding a Data Structures assignment.")  
  .font(.title2)  
  .multilineTextAlignment(.leading)  
}  
.navigationTitle("About Us")  
}  
}
```



Code along



Traffic Light

Part I



Start the traffic light

Our aim is to simulate the traffic light. We want to navigate to red screen, from red we want to navigate to yellow screen and then from yellow to green,

Traffic Light

RedLightView



```
struct RedLightView: View {  
    var body: some View {  
        ZStack {  
            Color.red  
                .ignoresSafeArea()  
        }  
        .navigationTitle("Red Light")  
    }  
}
```

- Create red, yellow and green light views
- Navigate from Content view to Red Light view

Traffic Light

Content View



```
struct ContentView: View {  
    var body: some View {  
        NavigationStack {  
            NavigationLink("Start the traffic light") {  
                RedLightView()  
            }  
        }  
    }  
}
```

- 1 Navigation stack embedded in Content View to navigate to Red Light View

Traffic Light

RedLightView to YellowLightView



```
struct RedLightView: View {  
    var body: some View {  
        Color  
            .red  
            .ignoresSafeArea()  
    }  
}
```

```
struct YellowLightView: View {  
    var body: some View {  
        Color  
            .yellow  
            .ignoresSafeArea()  
    }  
}
```

- How to navigate from Red to Yellow ?
- Need a Navigation Stack inside Red Light View ? **No.**

Traffic Light

RedLightView



```
struct RedLightView: View {
    var body: some View {
        ZStack {
            Color.red
                .ignoresSafeArea()
            NavigationLink("Go to yellow", destination: YellowLightView())
        }
        .navigationTitle("Red Light")
    }
}
```

- No Navigation Stack needed inside Red Light View as we embedded one in Content View.

Traffic Light

YellowLightView to GreenLightView



```
struct YellowLightView: View {  
    var body: some View {  
        ZStack {  
            Color  
                .yellow  
                .ignoresSafeArea()  
        }  
    }  
}
```

```
struct GreenLightView: View {  
    var body: some View {  
        ZStack {  
            Color  
                .green  
                .ignoresSafeArea()  
        }  
    }  
}
```

- Same to navigate from Yellow to Green
- Only 1 Navigation Stack at a root view enough to navigate till any number of views in the same information hierarchy.

Traffic Light

YellowLightView



```
struct YellowLightView: View {
    var body: some View {
        ZStack {
            Color.yellow
                .ignoresSafeArea()
            NavigationLink("Go to green", destination: GreenLightView())
        }
        .navigationTitle("Yellow Light")
    }
}
```

Traffic Light

GreenLightView



```
struct GreenLightView: View {  
    var body: some View {  
        ZStack {  
            Color.green  
                .ignoresSafeArea()  
        }  
        .navigationTitle("Green Light")  
    }  
}
```



The app should work in simulator as shown in the video.



Activity



```
struct ProfileView: View {
  var body: some View {
    NavigationStack {
      List {
        NavigationLink("About Us", destination: AboutUsView())
      }
      .navigationTitle("Profile")
      .navigationBarTitleDisplayMode(.inline)
    }
  }
}
```

```
struct AboutUsView: View {
  var body: some View {
    List {
      ZStack {
        Color.mint
        Image(systemName: "graduationcap.fill")
          .font(.system(size: 60))
      }
    }
  }
}
```



```

    }
    .frame(height: 200)
    .padding()

    Text("We are a team of 3 developers who met in a college canteen while avoiding a Data Structures assignment.")
        .font(.title2)
        .multilineTextAlignment(.leading)
    Section {
        NavigationLink("John Doe", destination: TeamMemberView())
    } header: {
        Text("Team members")
    }
}
.navigationTitle("About Us")
}
}

```

```

struct TeamMemberView: View {
    var memberName:String = "John Doe"
    var memberDescription: String = "Illogical encyclopedia"
    var memberEmoji:String = "🤔"
    var body: some View {
        Text(memberEmoji)
            .font(.system(size: 100))
        Text(memberName)
            .font(.largeTitle)
        Text(memberDescription)
            .font(.subheadline)
    }
}

```


Navigation Link

Customizing links

```
| NavigationLink(destination: () -> View ,  
                  label: () -> View )
```



```
| NavigationLink {  
    code  
} label: {  
    code  
}
```

1st API for Navigation link we are studying in this session is the one that takes arguments **title** & **destination**.

The 1st api only allows text as links. In order to navigate through images/cards/view compositions etc we introduce the 2nd API.

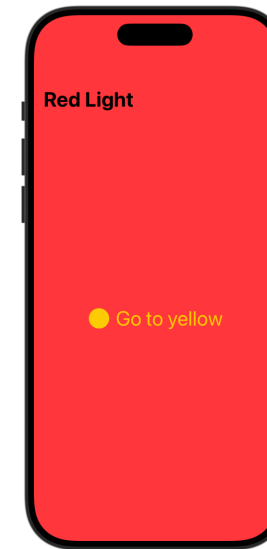
2nd API for Navigation link we are studying in this session is the one that takes arguments **destination** & **label**

This allows us to customize the links through which the navigation is taking place

Navigation Link

Customizing links

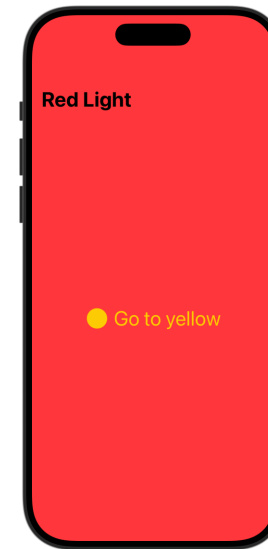
```
NavigationLink(destination: YellowLightView()) {  
    HStack {  
        Image(systemName: "circle.fill")  
        Text("Go to yellow")  
    }  
    .foregroundColor(.yellow)  
    .font(.largeTitle)  
}
```



Navigation Link

Customizing links

```
NavigationLink {  
  YellowLightView()  
} label: {  
  HStack {  
    Image(systemName: "circle.fill")  
    Text("Go to yellow")  
  }  
  .foregroundColor(.yellow)  
  .font(.largeTitle)  
}
```

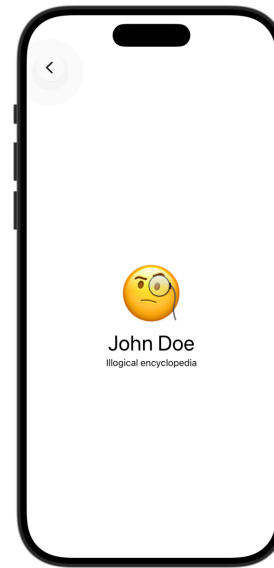
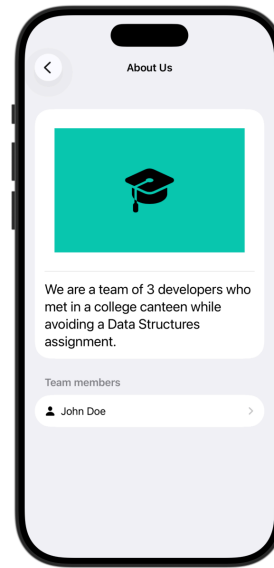
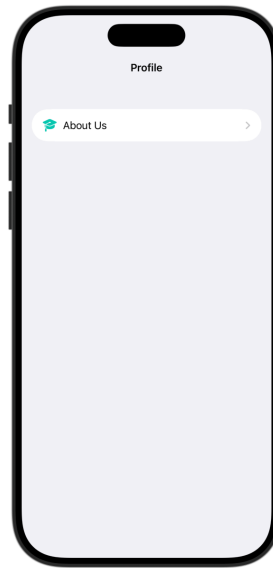


Both arguments passed as closures.



Activity

Profile



```
struct ProfileView: View {
    var body: some View {
        NavigationStack {
            List {
                NavigationLink {
                    AboutUsView()
                } label: {
                    Image(systemName: "graduationcap.fill")
                        .foregroundColor(.mint)
                    Text("About Us")
                }
            }
        }
        .navigationTitle("Profile")
        .navigationBarTitleDisplayMode(.inline)
    }
}
```

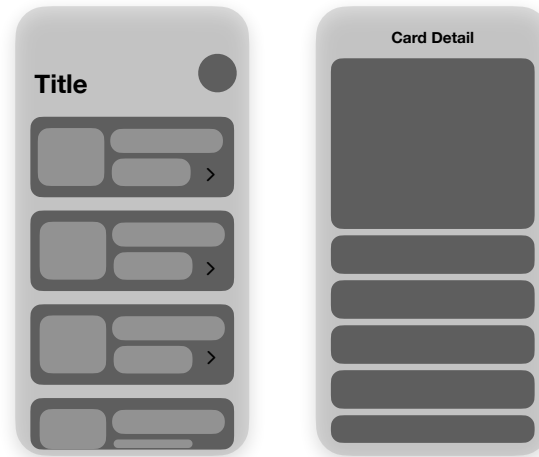
```
struct AboutUsView: View {
```

```
var body: some View {
    List {
        ZStack {
            Color.mint
            Image(systemName: "graduationcap.fill")
                .font(.system(size: 60))
        }
        .frame(height: 200)
        .padding()

        Text("We are a team of 3 developers who met in a college canteen while avoiding a Data Structures assignment.")
            .font(.title2)
            .multilineTextAlignment(.leading)
        Section {
            NavigationLink {
                TeamMemberView()
            } label: {
                Image(systemName: "person.fill")
                Text("John Doe")
            }

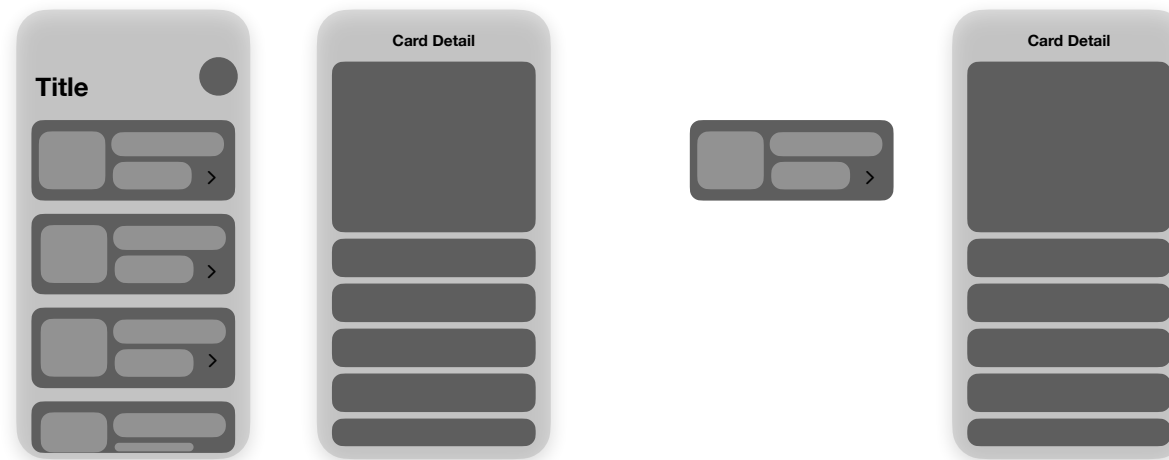
            } header: {
                Text("Team members")
            }
        }
        .navigationTitle("About Us")
    }
}
```

CardView navigate to CardDetailView



Many apps have screens that list some elements (books, notes, dishes) etc as list of cards and when any of the cards are tapped they navigate to a detail view for the tapped element. All the cards lead to the same view but with different content.

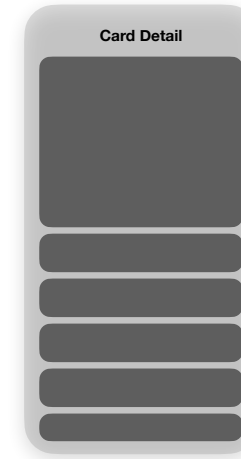
CardView navigate to CardDetailView



Such scenarios re-use views for the Card and CardDetail view, since both are generally a composition of views hence provide modularity. A CardView acts as a custom navigation link for the navigating to the destination of CardDetailView for the particular element in the collection.

CardView navigate to CardDetailView

```
ForEach(elements) { element in
  NavigationLink {
    CardDetailView(element)
  } label: {
    CardView(element)
  }
}
```



This is where foreach and navigation link can work together as shown



Code along

Notes App

Note structure

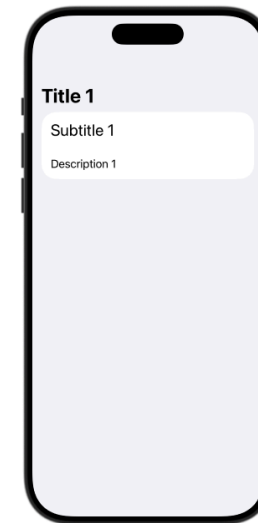


```
struct Note: Identifiable {  
    var id: String {  
        title + subtitle + description  
    }  
    var title: String  
    var subtitle: String  
    var description: String  
    static let sampleNotes: [Note] = [  
        Note(title: "Title 1", subtitle: "Subtitle 1", description: "Description 1"),  
        Note(title: "Title 2", subtitle: "Subtitle 2", description: "Description 2"),  
        Note(title: "Title 3", subtitle: "Subtitle 3", description: "Description 3"),  
        Note(title: "Title 4", subtitle: "Subtitle 4", description: "Description 4"),  
        Note(title: "Title 5", subtitle: "Subtitle 5", description: "Description 5"),  
    ]  
}
```

Notes App

Note Detail View

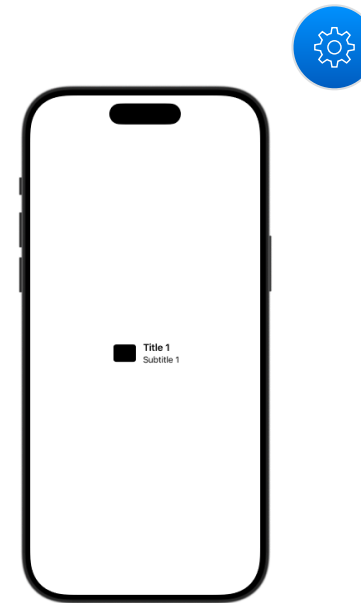
```
struct NoteDetailView: View {  
    var note: Note  
    var body: some View {  
        List {  
            Section {  
                Text(note.subtitle)  
                    .font(.title)  
                Text(note.description)  
                    .font(.title3)  
                    .multilineTextAlignment(.leading)  
            }  
            .listRowSeparator(.hidden)  
        }  
        .navigationTitle(note.title)  
    }  
}
```



Notes App

Note Row View

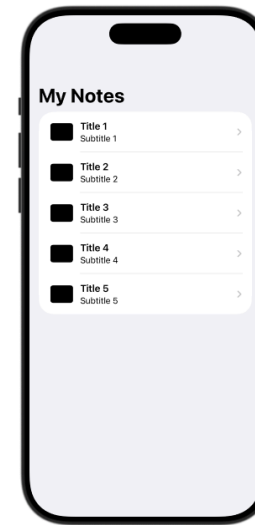
```
struct NoteRowView: View {  
    var note: Note  
    var body: some View {  
        HStack {  
            Image(systemName: "rectangle.fill")  
                .font(.largeTitle)  
            VStack(alignment: .leading) {  
                Text(note.title)  
                    .font(.headline)  
                Text(note.subtitle)  
                    .font(.subheadline)  
            }  
        }  
    }  
}
```



Notes App

Note List View

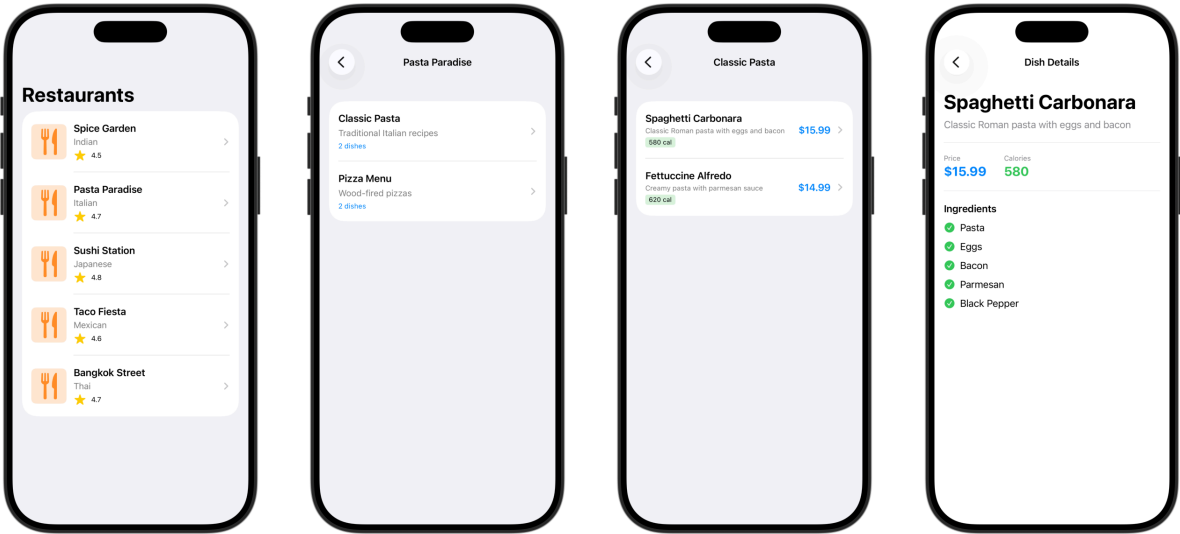
```
struct NoteListView: View {
    var notes = Note.sampleNotes
    var body: some View {
        NavigationStack {
            List {
                ForEach(notes) { note in
                    NavigationLink {
                        NoteDetailView(note: note)
                    } label: {
                        NoteRowView(note: note)
                    }
                }
            }
        }
        .navigationTitle("My Notes")
    }
}
```





Activity

Restaurant App hierarchy



Restaurant App hierarchy



- Starter project has all the views and the data.
- Less than 20 lines of code needed to make the project run as intended.